

Software Requirements Specification (SRS)

Project: Student Attendance Management System
Version: 1.0
Authors: Piyush A Patel; Neel Chandrakar; Nookala Sai Tushar Krishna; Pranav Somashekar
Date: 03-09-2026
Status: Draft — For Submission

Project Team

USN	Name
PES2UG24CS350	Piyush A Patel
PES2UG24CS310	Neel Chandrakar
PES2UG24CS331	Nookala Sai Tushar Krishna
PES2UG24CS364	Pranav Somashekar

Revision History

Version	Date	Author	Change summary	Approval
1.0	03-09-2026	Piyush A Patel	Initial SRS draft for SAMS, aligned to course SRS template	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features (Detailed)
- 5. Non-Functional Requirements (Detailed)
- 6. Quality Attributes & Acceptance Tests
- 7. System Models and Diagrams
- 8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Student Attendance Management System (SAMS). It defines functional and non-functional requirements, interfaces, and verification criteria intended for the project guide, evaluators, and the student development team to use as a reference during design and implementation.

1.2 Scope

Covers user interactions at the web application for three roles — Administrator, Faculty, and Student — including master-data management, attendance marking, automatic percentage calculation, and reporting. Excludes external ERP integration, biometric/facial-recognition capture, mobile applications, and cloud deployment, none of which are part of the current project requirements.

1.3 Audience

Project guide/evaluator, student developers (project team), and any future contributor who extends or maintains the system after the initial submission.

1.4 Definitions

List of acronyms: SRS, SAMS, Admin, CRUD, DB, UI, ORM, FR, NFR, RTM.

Term	Description
SAMS	Student Attendance Management System (this project)
Admin	Administrator — the user role with full system access
Section	A subdivision of a class/batch, e.g. "CSE-3A"
Attendance %	Ratio of classes attended to classes held, expressed as a percentage
Low Attendance	Attendance percentage below the configured minimum threshold (default 75%)
ORM	Object-Relational Mapping (used by Django to interact with SQLite)
RTM	Requirements Traceability Matrix

2. Overall Description

2.1 Product Perspective

SAMS is a new, standalone web application, not a modification of an existing system. It is a self-contained Django project with a single relational database (SQLite) and a server-rendered frontend (Django Templates, HTML, CSS, JavaScript), following a three-tier structure: presentation (templates/static assets), application logic (views, forms), and data (Django ORM models over SQLite).

2.2 Major Product Functions (Detailed)

- Authenticate user and enforce role-based access
- Manage students, faculty, sections, and subjects (Administrator)

- Assign faculty to subject-section combinations
- Mark and edit attendance for assigned subjects (Faculty)
- Automatic subject-wise and overall attendance percentage calculation
- Low-attendance identification against a configurable threshold
- Attendance reports and attendance history (all roles, scoped by permission)
- Role-specific dashboards

2.3 User Roles and Characteristics (Expanded)

- Administrator: single or small set of privileged users (e.g. HOD/office staff); comfortable with basic web forms; requires full CRUD access to master data and system-wide reports.
- Faculty: teaching staff with limited technical background; needs a fast interface for marking attendance per class period, and read access to reports for their own subjects.
- Student: largest user group; requires only read-only access to personal attendance data; interface must require no training.

2.4 Operating Environment

- Backend: Python 3.x with the Django web framework.
- Database: SQLite (file-based, bundled with Django by default).
- Frontend: Django Template Language rendering HTML5, styled with CSS and vanilla JavaScript.
- Server: Django development server for evaluation; optionally served via WSGI on a local network for demonstration.
- Client: any modern desktop/laptop browser (Chrome, Firefox, Edge) with JavaScript enabled.

2.5 Constraints

- Must be implemented in Python/Django, per course requirements; SQLite is the only database, so no database-specific stored procedures.
- Frontend must use Django Templates rather than a separate SPA framework.
- Development and evaluation are confined to a single academic semester, limiting scope to core attendance-management functionality.
- No biometric, facial-recognition, AI/ML-based, or cloud-hosted components, consistent with the project's academic scope.

3. External Interface Requirements

3.1 User Interfaces

Primary UI: browser-rendered pages with a clear navigation structure common to all roles (login page redirecting to a role-specific dashboard). Usability: plain-language forms and validation messages; consistent layout via a shared base template, navigation bar, and footer.

3.2 Hardware Interfaces

SAMS has no dedicated hardware peripherals. Client-side hardware requirements are limited to a standard desktop, laptop, or mobile device with a keyboard/mouse or touchscreen, capable of running a modern web browser. No card readers, biometric scanners, or specialised input devices are required.

3.3 Software Interfaces

- Django Framework: request/response handling, ORM, authentication, and templating used throughout the system.
- SQLite: accessed exclusively through the Django ORM; no direct SQL queries required for standard operations.
- Web Browser: client-side interface is rendered HTML/CSS/JavaScript, requiring no additional client software.

3.4 Communications

- Standard HTTP/HTTPS between browser and Django server; session-based authentication using signed cookies.
- All state-changing requests (forms) are protected by Django's CSRF token mechanism.

Note: This SRS defines 40 functional requirements, 6 non-functional requirements, 3 security objectives, and 5 security requirements across Sections 4 and 5, exceeding the minimum of 15 FRs, 5 NFRs, 2 security objectives, and 5 security requirements.

4. System Features (Detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow SAMS-F-###.

4.1 Authentication

Description: Authenticate registered users (Administrator, Faculty, Student) against stored credentials and maintain a secure session.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-001	The system shall allow a registered user to log in using a username and password.	Functional	High	Admin/Faculty/Student	AC-001: Valid credentials lead to an authenticated session. Test: TC-Auth-01	Uses Django auth
SAMS-F-002	The system shall validate credentials and deny access with an error message on failure.	Functional	High	Security	AC-002: Invalid login shows error, no session created. Test: TC-Auth-02	
SAMS-F-003	The system shall maintain a session until logout or expiry.	Functional	High	Security	AC-003: Session persists across pages until logout/timeout. Test: TC-Auth-03	Django session framework
SAMS-F-004	The system shall allow a logged-in user to log out, terminating the session.	Functional	High	Admin/Faculty/Student	AC-004: Logout invalidates session; protected pages redirect to login. Test: TC-Auth-04	

4.2 Role-Based Access Control

Description: Restrict pages and actions according to the user's assigned role (Administrator, Faculty, Student).

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-005	The system shall assign every user account exactly one role.	Functional	High	Admin	AC-005: Each user record has a single non-null role field. Test: TC-RBAC-01	
SAMS-F-006	The system shall restrict Administrator-only pages to Administrator accounts.	Functional	High	Security	AC-006: Direct URL access by non-admin is rejected. Test: TC-RBAC-02	Server-side check
SAMS-F-007	The system shall restrict attendance-marking pages to Faculty, and only for assigned subjects/sections.	Functional	High	Security	AC-007: Faculty cannot mark for unassigned sections. Test: TC-RBAC-03	
SAMS-F-008	The system shall restrict Student accounts to read-only views of their own data.	Functional	High	Security	AC-008: Student cannot view another student's records. Test: TC-RBAC-04	

4.3 Student Management

Description: Administrator CRUD operations on student master data.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-009	The system shall allow the Administrator to add a student record (name, USN, section, semester, contact).	Functional	High	Admin	AC-009: New record saved and retrievable. Test: TC-Stu-01	
SAMS-F-010	The system shall allow the Administrator to update existing student details.	Functional	High	Admin	AC-010: Edited fields persist. Test: TC-Stu-02	
SAMS-F-011	The system shall allow the Administrator to deactivate or remove a student record.	Functional	Medium	Admin	AC-011: Deactivated student cannot log in. Test: TC-Stu-03	
SAMS-F-012	The system shall prevent duplicate student records with the same USN.	Functional	High	Data Integrity	AC-012: Duplicate USN submission is rejected. Test: TC-Stu-04	DB unique constraint

4.4 Faculty Management

Description: Administrator CRUD operations on faculty master data and subject-section assignment.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-013	The system shall allow the Administrator to add a faculty record (name, employee ID, department, contact).	Functional	High	Admin	AC-013: New faculty record saved. Test: TC-Fac-01	
SAMS-F-014	The system shall allow the Administrator to update or deactivate a faculty record.	Functional	Medium	Admin	AC-014: Edits/deactivation persist. Test: TC-Fac-02	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-015	The system shall allow the Administrator to assign one or more subject-section combinations to a faculty member.	Functional	High	Admin	AC-015: Assignment appears on faculty dashboard. Test: TC-Fac-03	

4.5 Class and Section Management

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-016	The system shall allow the Administrator to create a section (name, semester, academic year).	Functional	High	Admin	AC-016: New section available for student assignment. Test: TC-Cls-01	
SAMS-F-017	The system shall allow the Administrator to update or delete a section, provided no attendance records would be orphaned.	Functional	Medium	Admin	AC-017: Delete blocked if dependent records exist. Test: TC-Cls-02	
SAMS-F-018	The system shall allow students to be assigned to exactly one section per academic term.	Functional	High	Data Integrity	AC-018: Student record references exactly one section_id. Test: TC-Cls-03	

4.6 Subject Management

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-019	The system shall allow the Administrator to create a subject (name, subject code, semester).	Functional	High	Admin	AC-019: New subject saved. Test: TC-Sub-01	
SAMS-F-020	The system shall allow the Administrator to map a subject to one or more sections.	Functional	High	Admin	AC-020: Mapping visible in subject list. Test: TC-Sub-02	
SAMS-F-021	The system shall allow the Administrator to update or remove a subject record.	Functional	Medium	Admin	AC-021: Edits/removal persist. Test: TC-Sub-03	

4.7 Attendance Management

Description: Faculty marking and editing of attendance for assigned subject-section combinations.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-022	The system shall allow Faculty to select an assigned	Functional	High	Faculty	AC-022: Marking page loads correct student roster. Test: TC-Att-01	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
	subject-section and a date to mark attendance.					
SAMS-F-023	The system shall display enrolled students for marking as Present or Absent.	Functional	High	Faculty	AC-023: All section students listed with two-state control. Test: TC-Att-02	
SAMS-F-024	The system shall save one attendance record per student, per subject, per date, linked to the marking faculty.	Functional	High	Data Integrity	AC-024: Record count = roster size after submission. Test: TC-Att-03	
SAMS-F-025	The system shall prevent duplicate marking for the same subject/section/date and instead allow editing.	Functional	High	Data Integrity	AC-025: Re-submission updates, not duplicates, records. Test: TC-Att-04	DB unique constraint
SAMS-F-026	The system shall allow Faculty to edit attendance already marked, within an authorized correction window.	Functional	Medium	Faculty	AC-026: Edit permitted within window, blocked after. Test: TC-Att-05	Configurable window
SAMS-F-027	The system shall reject marking for dates outside the current term or for unassigned subjects/sections.	Functional	High	Security	AC-027: Out-of-term/unassigned submission returns error. Test: TC-Att-06	

4.8 Attendance Calculation

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-028	The system shall calculate subject-wise attendance % as $(\text{attended} \div \text{held}) \times 100$.	Functional	High	Business Rule	AC-028: Percentage matches manual calculation on sample data. Test: TC-Calc-01	
SAMS-F-029	The system shall calculate overall attendance % across all subjects for the current term.	Functional	High	Business Rule	AC-029: Overall % matches aggregate of subject-wise values. Test: TC-Calc-02	
SAMS-F-030	The system shall recalculate percentages automatically whenever attendance data changes.	Functional	High	Business Rule	AC-030: Percentage updates immediately after edit. Test: TC-Calc-03	

4.9 Attendance Reports

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-031	The system shall allow Faculty to generate a subject-wise report per section, listing each student's %.	Functional	High	Faculty	AC-031: Report lists all section students with correct %. Test: TC-Rep-01	

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-032	The system shall allow the Administrator to generate section-wise and subject-wise reports institution-wide.	Functional	High	Admin	AC-032: Report aggregates across sections. Test: TC-Rep-02	
SAMS-F-033	The system shall allow a Student to view their own subject-wise and overall report only.	Functional	High	Security	AC-033: Student report scoped to own student_id. Test: TC-Rep-03	
SAMS-F-034	The system shall display date-wise attendance history for a selected student and subject.	Functional	High	Admin/Faculty/Student	AC-034: History lists Present/Absent per date. Test: TC-Rep-04	

4.10 Low-Attendance Monitoring

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-035	The system shall allow the Administrator to configure a minimum attendance threshold (default 75%).	Functional	Medium	Admin	AC-035: Threshold value editable and persisted. Test: TC-Low-01	
SAMS-F-036	The system shall identify students whose % falls below the configured threshold.	Functional	High	Business Rule	AC-036: List matches students below threshold on sample data. Test: TC-Low-02	
SAMS-F-037	The system shall allow Faculty and Administrator to view the low-attendance list for their respective scope.	Functional	High	Faculty/Admin	AC-037: Faculty sees own sections; Admin sees all. Test: TC-Low-03	

4.11 Dashboard

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments/Dependencies
SAMS-F-038	The system shall present an Administrator dashboard summarising totals and low-attendance count.	Functional	Medium	Admin	AC-038: Dashboard counts match underlying data. Test: TC-Dash-01	
SAMS-F-039	The system shall present a Faculty dashboard summarising assigned subjects and today's marking status.	Functional	Medium	Faculty	AC-039: Status reflects whether today's attendance is marked. Test: TC-Dash-02	
SAMS-F-040	The system shall present a Student dashboard summarising overall % and subjects below threshold.	Functional	Medium	Student	AC-040: Dashboard values match report data. Test: TC-Dash-03	

5. Non-Functional Requirements (Detailed)

NFRs below are measurable and tied to test plans. IDs follow SAMS-NF-####.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SAMS-NF-001	Standard pages (dashboard, attendance list, report) shall load within 3 seconds under normal load for a database of up to a few thousand attendance records.	Performance	High	Page load timed at ≤ 3 s in local test. Test: TC-Perf-01
SAMS-NF-002	Once saved, attendance records shall persist correctly and be retrievable in reports without data loss.	Reliability	High	Sample records verified present after restart. Test: TC-Rel-01
SAMS-NF-003	The user interface shall use consistent navigation, labelling, and layout across all pages for a given role.	Usability	Medium	UI review checklist pass. Test: TC-UX-01
SAMS-NF-004	The codebase shall follow Django's standard project structure with reusable model methods for calculations.	Maintainability	Medium	Code review confirms no duplicated calculation logic. Test: TC-Maint-01
SAMS-NF-005	The data model shall be normalized so migration to a larger database engine would not require a schema redesign.	Scalability	Low	Schema review against 3NF checklist. Test: TC-Scal-01
SAMS-NF-006	The system shall validate all form inputs and reject invalid data with a clear, field-level error message.	Reliability	High	Invalid input (e.g., future-dated attendance) is rejected with message. Test: TC-Rel-02

5.1 Security

5.1.1 Security Objectives

- Protect the confidentiality of user credentials at rest and in transit.
- Enforce role-based authorization on the server side for every protected action, not only in the UI.
- Preserve the integrity and accountability of attendance records once marked.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SAMS-SR-001	User passwords shall be stored using Django's built-in password hashing; plain-text passwords shall never be stored.	Security	High	AC-SR-001: DB inspection shows hashed values only. Test: TC-Sec-01
SAMS-SR-002	All pages that display or modify data shall require an authenticated session.	Security	High	AC-SR-002: Unauthenticated request redirects to login. Test: TC-Sec-02
SAMS-SR-003	Role-based checks shall be enforced server-side in Django views, not only hidden in the frontend.	Security	High	AC-SR-003: Direct URL request by wrong role is rejected. Test: TC-Sec-03

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SAMS-SR-004	All state-changing forms shall be protected against Cross-Site Request Forgery using Django's CSRF tokens.	Security	High	AC-SR-004: Form submission without valid CSRF token is rejected. Test: TC-Sec-04
SAMS-SR-005	All database queries shall go through the Django ORM to prevent SQL injection.	Security	High	AC-SR-005: No raw string-concatenated SQL in codebase (code review). Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no critical NFR failures, and the RTM (Section 8) shows all listed test cases passed.

Acceptance test suites: Authentication & Access Control, Student/Faculty/Subject/Section Management, Attendance Marking & Editing, Attendance Calculation & Reports, Low-Attendance Monitoring, Security.

7. System Models and Diagrams

7.1 UML Use-Case Diagram

Two use-case diagrams are provided below: one for the Administrator, and one covering Faculty and Student, reflecting the three actors of the system.

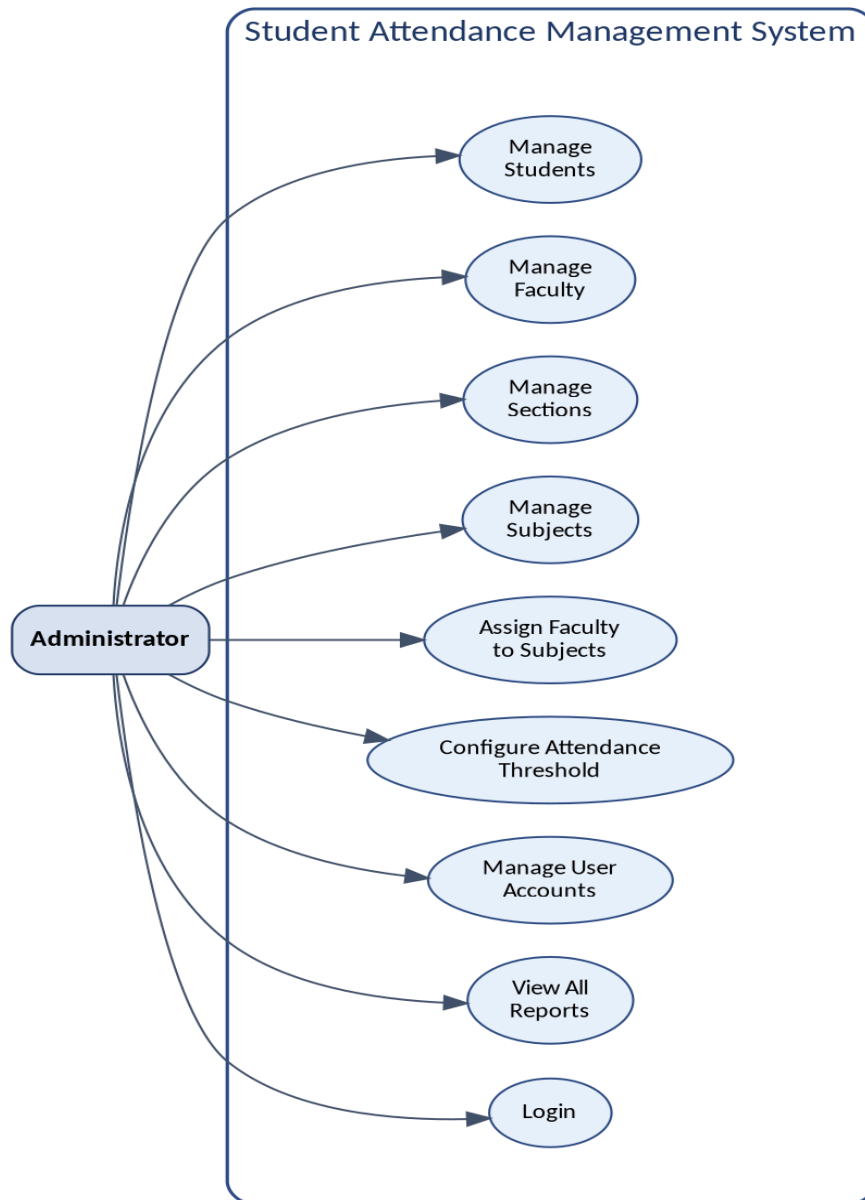


Figure 7.1 — Administrator Use-Case Diagram

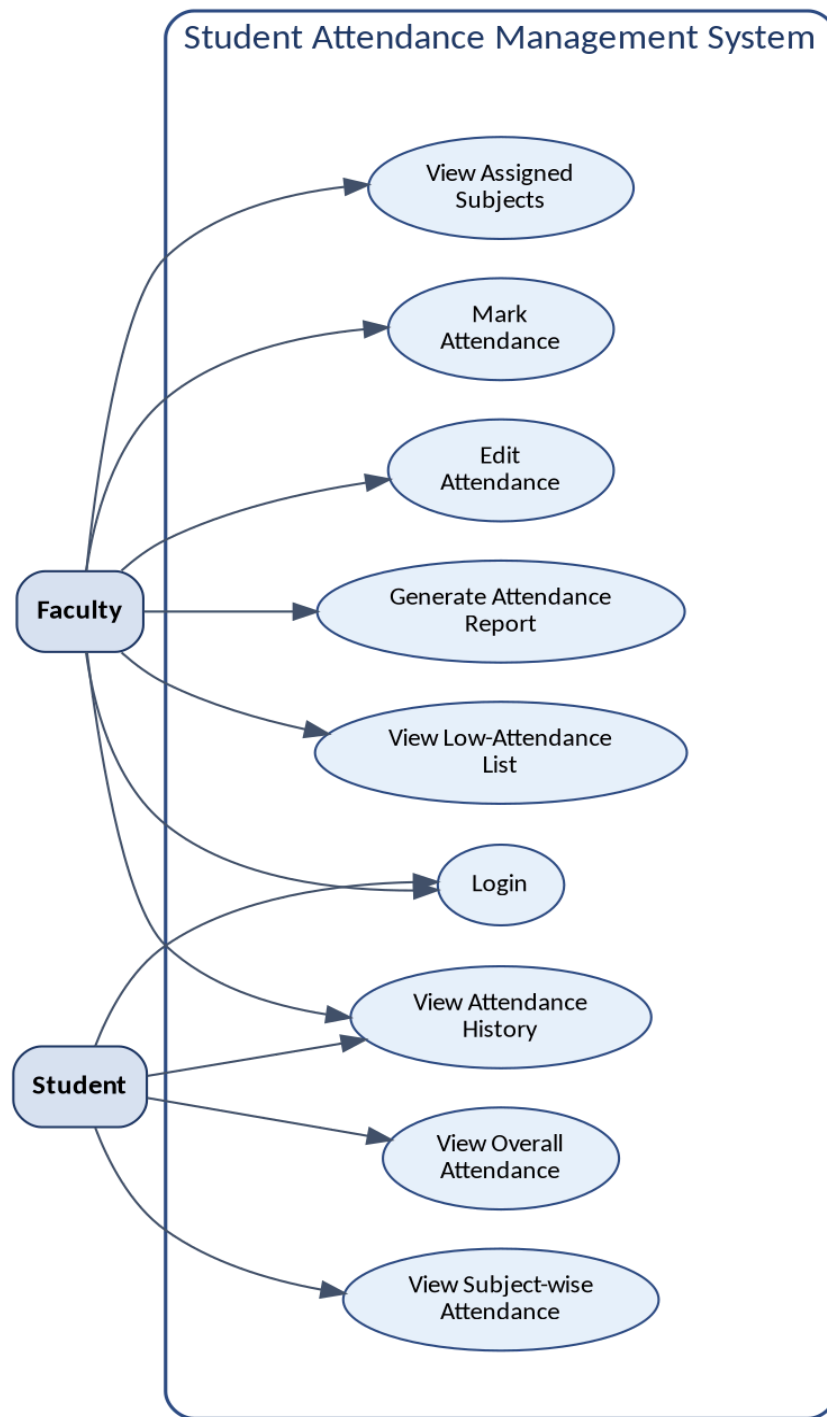


Figure 7.2 — Faculty and Student Use-Case Diagram

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status(N /P/A)	Comments
SAMS-F-001	Login authentication	4.1 / DS-Auth-01	AuthModule	TC-Auth-01	N	

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status(N/P/A)	Comments
SAMS-F-007	Restrict marking to assigned faculty	4.2 / DS-RBAC-01	AuthModule	TC-RBAC-03	N	
SAMS-F-012	Prevent duplicate USN	4.3 / DS-Stu-01	StudentModule	TC-Stu-04	N	
SAMS-F-022	Mark attendance for section	4.7 / DS-Att-01	AttendanceModule	TC-Att-01	N	
SAMS-F-025	Prevent duplicate attendance	4.7 / DS-Att-02	AttendanceModule	TC-Att-04	N	
SAMS-F-028	Subject-wise % calculation	4.8 / DS-Calc-01	ReportModule	TC-Calc-01	N	
SAMS-F-036	Low-attendance identification	4.10 / DS-Low-01	ReportModule	TC-Low-02	N	
SAMS-NF-001	Page load performance	5 / DS-Perf-01	WebUI	TC-Perf-01	N	
SAMS-NF-006	Form input validation	5 / DS-Rel-02	Forms	TC-Rel-02	N	
SAMS-SR-001	Password hashing	5.1.2 / DS-Sec-01	AuthModule	TC-Sec-01	N	
SAMS-SR-003	Server-side role enforcement	5.1.2 / DS-Sec-03	AuthModule	TC-Sec-03	N	
SAMS-SR-004	CSRF protection	5.1.2 / DS-Sec-04	Forms	TC-Sec-04	N	

Status legend: N = Not yet tested, P = Passed, A = Failed/Action needed. To be updated during the testing phase.

Appendices (Supplementary)

The following appendices are not part of the 8 core sections required by the course SRS template. They are included as additional supporting material relevant to the DBMS/Software Engineering scope of this project.

Appendix A — Database Schema Reference

Entity	Key Attributes
User	user_id (PK), username, password (hashed), role, email
Student	student_id (PK), user_id (FK), USN, name, section_id (FK), semester, contact_no
Faculty	faculty_id (PK), user_id (FK), employee_id, name, department, contact_no
Section	section_id (PK), section_name, semester, academic_year
Subject	subject_id (PK), subject_code, subject_name, semester
Subject_Section	id (PK), subject_id (FK), section_id (FK)
Faculty_Assignment	id (PK), faculty_id (FK), subject_id (FK), section_id (FK)
Attendance	attendance_id (PK), student_id (FK), subject_id (FK), faculty_id (FK), date, status, marked_at, modified_at
Attendance_Config	id (PK), threshold_percentage

- User (1)—(1) Student/Faculty; Section (1)—(M) Student.
- Subject (M)—(M) Section via Subject_Section; Faculty (M)—(M) Subject/Section via Faculty_Assignment.
- Student (1)—(M), Subject (1)—(M), Faculty (1)—(M) Attendance.

Appendix B — Additional System Models

B.1 Data Flow Diagram Description

At Level 0, the system exchanges data with three external entities — Administrator, Faculty, Student. At Level 1: (1) Master Data Management, (2) Authentication and Access Control, (3) Attendance Recording, (4) Report and Percentage Generation.

B.2 Entity Relationship Description

Core entities — User, Student, Faculty, Section, Subject, Attendance — relate as summarised in Appendix A.

B.3 Activity Diagram Description

1. Faculty logs in and is directed to their dashboard.
2. Faculty selects subject, section, and date.
3. System checks whether attendance already exists for that combination.
4. Faculty marks each student Present/Absent and submits.
5. System validates, saves records, and recalculates percentages.
6. System displays confirmation and returns to dashboard.

Appendix C — Future Scope

- Email/SMS notifications for low attendance.
- Export of reports to PDF/Excel.
- Responsive/mobile-friendly interface.
- Migration from SQLite to PostgreSQL for larger deployments.
- QR-code-based attendance marking as a faster manual alternative.